Assigment - 13.2

V.Vishnu Dattu
2303A52035
Batch no. : 38

Task Description -1 (Refactoring – Eliminating Repetitive Logic)
• Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.
Instructions:
• Ask AI to locate repeated computations within the program.
• Encapsulate repeated logic into reusable functions.
• Verify that program output remains unchanged after refactoring.
• Add proper docstrings to all defined functions.
Sample Code:
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
Expected Output:
A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

Code:

```
length = 8
"""
Calculate and display the area and perimeter of a rectangle.
Args:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.
Returns:
    None
Examples:
    >>> print_rectangle_properties(8, 4)
    Area: 32
    Perimeter: 24
    >>> print_rectangle_properties(12, 6)
    Area: 72
    Perimeter: 36
"""
```

```
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

"""locate repeated code and refactor it into a function to
improve code reusability and maintainability."""
def print_rectangle_properties(length, width):
    print("Area:", length * width)
    print("Perimeter:", 2 * (length + width))
```

```
print_rectangle_properties(8, 4)
print_rectangle_properties(12, 6)
```

Output:

```
PS C:\Users\VR\Documents\AI_Assisted_Coding> & C:/Users/VR/AppData/Local/Py
thon/pythoncore-3.14-64/python.exe "c:/Users/VR/Documents/AI_Assisted_Codin
g/lab test-1/lab 9/lab 12/lab 13/perimeter.py"
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
PS C:\Users\VR\Documents\AI_Assisted_Coding>
```

Task Description -2 (Refactoring – Modularizing Inline Calculations)

• Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Instructions:

• Identify repeated calculation patterns in the code.

• Convert them into reusable functions.

• Ensure readability and proper documentation.

Sample Code:

amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)

Expected Output:
A modular Python program that uses a reusable function to compute final amounts for different inputs.

Code:

```
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)

"""• Identify repeated calculation patterns in the code.
• Convert them into reusable functions.
• Ensure readability and proper documentation for the
functions."""
def calculate_final_amount(amount):
    """Calculate the final amount after applying a 10%
discount.

    Args:
        amount (float): The original amount before discount.

    Returns:
        float: The final amount after applying the discount.
    """
    discount = amount * 0.10
    return amount - discount
```

```
print("Final Amount:", calculate_final_amount(1000))
print("Final Amount:", calculate_final_amount(2000))
```

Output:

```
PS C:\Users\VR\Documents\AI_Assisted_Coding> & C:/Users/VR/AppDat
a/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/VR/Documen
ts/AI_Assisted_Coding/lab test-1/lab 9/lab 12/lab 13/amount.py"
Final Amount: 900.0
Final Amount: 1800.0
Final Amount: 900.0
Final Amount: 1800.0
PS C:\Users\VR\Documents\AI_Assisted_Coding>
```

Task Description -3 (Refactoring – Improving Loop and

Conditional Performance)
• Task: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.
Instructions:
• Use AI to recommend more efficient data structures or logic simplifications.
• Replace inefficient constructs while preserving program behavior.
• Compare execution efficiency before and after refactoring.
Sample Code:
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
exists = False
for student in students:
if name == student:
exists = True
if exists:
print(name, "found")
else:
print(name, "not found")
Expected Output:
An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
    exists = False
for student in students:
    if name == student:
        exists = True
if exists:
    print(name, "found")
else:
    print(name, "not found")

"""make the code more efficient and readable by using a
function to check for the presence of names in the students
list."""
def check_name(name, students):
    """Check if a name exists in the students list.

    Args:
        name (str): The name to check for.
        students (list): The list of student names.

    Returns:
```

```
        bool: True if the name exists in the list, False
otherwise.
    """
    return name in students
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
    if check_name(name, students):
        print(name, "found")
    else:
        print(name, "not found")
```

Output:

```
PS C:\Users\VR\Documents\AI_Assisted_Coding> & C:/Users/VR/AppDat
a/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/VR/Documen
ts/AI_Assisted_Coding/lab test-1/lab 9/lab 12/lab 13/check_list.p
y"
Bhavya found
Charan found
Esha not found
Bhavya found
PS C:\Users\VR\Documents\AI_Assisted_Coding>
```

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

• Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

• Detect hardcoded numeric values used directly in expressions.

• Define descriptive constants at the beginning of the program.

• Update calculations to use these constants without changing results.

Sample Code:

print("Simple Interest:", (1000 * 2 * 5) / 100)

print("Simple Interest:", (2000 * 2 * 5) / 100)

Expected Output:

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations..

Code:

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)

"""Calculate simple interest for different principal amounts,
rates, and time periods. declare a function to compute simple
```

```
interest and use it to calculate the interest for various
scenarios.
and also declare the attributes of the function in the
docstring for better understanding."""
def calculate_simple_interest(principal, rate, time):
    """Calculate simple interest based on the formula: (P * R
* T) / 100.

    Args:
        principal (float): The initial amount of money.
         rate (float): The annual interest rate (in percentage).
        time (float): The time the money is invested for (in
years).

    Returns:
        float: The calculated simple interest.
    """
    return (principal * rate * time) / 100
print("Simple Interest:", calculate_simple_interest(1000, 5,
2))
print("Simple Interest:", calculate_simple_interest(2000, 5,
2))
```

Output:

```
PS C:\Users\VR\Documents\AI_Assisted_Coding> & C:/Users/VR/AppDat
a/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/VR/Documen
ts/AI_Assisted_Coding/lab test-1/lab 9/lab 12/lab 13/Simple_inter
est.py"
Simple Interest: 100.0
Simple Interest: 200.0
Simple Interest: 100.0
Simple Interest: 200.0
PS C:\Users\VR\Documents\AI_Assisted_Coding>
```

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

• Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments..

Instructions:

• Replace non-descriptive variable names with meaningful identifiers.

• Add inline comments where the logic may be unclear.

• Ensure program output remains the same.

Sample Code:

x = 50

y = 30

z = x + y

print(z)

Expected Output:
A Python program with clear variable names and explanatory comments that improve readability without altering functionality.

Code:

```
x = 50
y = 30
z = x + y
print(z)
"""• Replace non-descriptive variable names with meaningful
identifiers.
• Add inline comments where the logic may be unclear.
• Ensure program output remains the same."""
def add_numbers(num1, num2):
    """Add two numbers and return the result.

    Args:
        num1 (float): The first number to be added.
        num2 (float): The second number to be added.

    Returns:
        float: The sum of the two numbers.
    """
    return num1 + num2
```

```
result = add_numbers(50, 30)
print(result)
```

Output:

```
PS C:\Users\VR\Documents\AI_Assisted_Coding> & C:/Users/VR/AppDat
a/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/VR/Documen
ts/AI_Assisted_Coding/lab test-1/lab 9/lab 12/lab 13/add.py"
80
80
PS C:\Users\VR\Documents\AI_Assisted_Coding>
```